01

Data Science ve Pandas

Odak doğrudan karmaşık modellere geçmeden önce veriyi tanımak, kalite kontrolü yapmak ve temel veri manipülasyonu alışkanlığı kazanmaktadır.

Çalışmalar Kaggle kaynakları ve Titanic train.csv veri seti üzerinden yapıldı.

Ne Anladım? Bir makine öğrenmesi modelinin başarısı doğrudan verinin temizliğine ve kalitesine bağlıdır. Bu nedenle çalışmaya başlamadan önce verinin boyutunu, eksikliklerini ve dağılımını inceledim.



Veri Yükleme ve Boyut İncelemesi

```python
import pandas as pd

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
dataset = pd.read_csv(url, index_col="PassengerId")

print("Veri seti boyutu:", dataset.shape)

# Veri çerçevesini (DataFrame) analiz için hazır hale getiriyoruz
# PassengerId sütununu satır indeksi yaptık (index_col="PassengerId")

df = dataset.copy()
# Burada csv dosyasını pandas dataframe'e aktardık
# Satırlar yolcuları, sütunlar ise yolculara ait özellikleri (Survived, Pclass, Name, Sex, Age vb.) tutuyor
print("İlk 5 gözlem:")
display(df.head())
```

İndeksleme: Tekil yolcu numarası olan PassengerId sütunu satır indeksi (index_col) olarak atanarak tablodaki gereksiz sütun fazlalığı önlenmiştir.

Boyut: Veri seti tam olarak 891 satır ve 11 sütundan oluşmaktadır.

Kopya Oluşturma (.copy()): Orijinal veri setini bellekte saf haliyle muhafaza etmek ve veri temizleme adımlarında olası bağlantı hatalarını engellemek amacıyla bağımsız bir kopya (df) oluşturulmuştur.

```
Veri seti boyutu: (891, 11)
İlk 5 gözlem:
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Veri Tipleri ve Eksik Veri Analizi

```
# Her sütunun veri tipini topluca kontrol eder
# (int64, float64, object/string veri tiplerini ayrıştırmak için gereklidi
print("Sütun veri tipleri: ")
print(df.dtypes)


# Her sütundaki eksik (NaN/Null) değer sayısını hesaplar
print("\n Eksik veri sayısı dağılımı:")
eksik_sayisi= df.isnull().sum()



# Yalnızca eksik verisi olan sütunları filtreler ve ekrana yazdırır
eksik_olanlar = eksik_sayisi[eksik_sayisi>0]
print(eksik_olanlar)



# Eksik verilerin yüzdelik oranını hesaplar (Karar vermeyi kolaylaştırır)
print("\n Eksik veri oranları: ")
eksik_oranı = (eksik_olanlar / len(df)) *100
print(eksik_oranı.round(2))

# Tüm veri setindeki toplam eksik hücre sayısını hesaplar
toplam_eksik = df.isnull().sum().sum()
print("\nToplam Eksik Değer Sayısı:", toplam_eksik)


# Titanic veri setinde 3 sütunda NaN değerler bulunur
# 1. Age (Yaş - ~%20 eksik): Medyan veya yolcu unvanlarına göre ortalamayla doldurulabilir
# 2. Cabin (Kabin - ~%77 eksik): Çok yüksek oranda boş olduğu için ya tamamen silinmeli ya da 'Kabin Var/Yok' şeklinde ikili bir özelliğe dö
# 3. Embarked (Biniş Limanı - 2 satır eksik): Çok az eksik olduğu için en sık tekrar eden limanla (Mod değeri, genellikle 'S') doldurulabili
```

```
Sütun veri tipleri:
Survived      int64
Pclass        int64
Name         object
Sex          object
Age         float64
SibSp         int64
Parch         int64
Ticket       object
Fare        float64
Cabin        object
Embarked     object
dtype: object

 Eksik veri sayısı dağılımı:
Age         177
Cabin       687
Embarked      2
dtype: int64

 Eksik veri oranları:
Age         19.87
Cabin       77.10
Embarked     0.22
dtype: float64

Toplam Eksik Değer Sayısı: 866
```

Veri Profili: Sayısal değişkenlerin (int64, float64) ve kategorik metinlerin (object) tipleri doğrulanmıştır.

Eksik Hücre Tespiti: 891 satırlık tabloda toplam 866 adet eksik hücre (NaN) tespit edilmiştir.

Temizleme Stratejisi:

- Cabin (687 Eksik - %77.10): Çok yüksek oranda boş olduğu için bilgi kaybı yaratmamak adına "Unknown" kategorisine dönüştürülmelidir.
- Age (177 Eksik - %19.87): Dağılımın merkezini korumak adına aykırı değerlerden etkilenmeyen medyan (28.0) ile doldurulacaktır.
- Embarked (2 Eksik - %0.22): Kayıp çok düşük olduğu için veri setinin mod ile tamamlanacaktır.

İstatistiksel Özetler ve Aykırı Değer Tespiti


```
#sayısal sütunların özet istatistiklerini hesaplar
print("\n Sayısal değişken istatistiksel özeti: ")
display(df.describe())

# Çıktı Yorumlama:
#Survived ortalama 0.38, yani yolcuların yaklaşık %38'i hayatta kalabilmiş.
# Age yaş ortalaması ~29.70'tir (bu değer 177 eksik değer hariç tutularak hesaplanır). Min 0.42 , max 80 yaşındadır.
#SibSp & Parch çoğu yolcunun tek başına seyahat ettiğini gösterir (%75 dilimi 0 1 arasındadır).
# Fare 50% 14.45 iken maksimum değer 512.33'tür. bu dağılım aykırı değerler olduğunu gösterir.

# Kategorik değişkenlerdeki sınıfların sayımı
print("\n Cinsiyet dağılımı: ")
print(df['Sex'].value_counts())

#kategorik değişkenlerdeki benzersiz (unique) sınıfların listesi
print("\n Biniş limanlarının benzersiz değerleri (Embarked): ")
print(df['Embarked'].unique())

#çıktı Yorumlama: ['S', 'C', 'Q', nan] sınıfları çıktıda yer alır
# dizideki 'nan' değeri bu sütundaki 2 adet eksik gözlemi temsil eder
```

Hayatta Kalma Dağılımı: Survived ortalaması 0.38, yani yolcuların yaklaşık %38'i hayatta kalmıştır.

Yaş Dağılımı: Yolcuların yaş ortalaması 29.70, medyanı ise 28.00'dir (Min: 0.42 bebek, Max: 80 yaş).

Aykırı Değer Tespiti: Bilet ücreti medyanı 14.45 iken maksimum değer 512.33 çıkmıştır. Çeyreklikler ile maksimum arasındaki bu uçurum aykırı değerleri gösterir.

Demografi: Gemide 577 erkek ve 314 kadın yolcu bulunmaktadır.

Sayısal değişken istatistiksel özeti:

	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
 Cinsiyet dağılımı:
Sex
male      577
female    314
Name: count, dtype: int64

 Biniş limanlarının benzersiz değerleri (Embarked):
['S' 'C' 'Q' nan]
```

Koşullu Seçim ve Filtreleme

```
# iloc ile ilk 5 satır ve ilk 3 sütunu konuma (indeks sırasına) göre seçer
print("\n iloc ile ilk 5 satır 3 sütun:")
display(df.iloc[:5, :3])

# loc ile çoklu şartlı filtreleme
# Pclass == 1 de seyahat eden ve hayatta kalan yolcuları listeler
kurtulan_birinci_sinif = df.loc[(df['Pclass'] == 1) & (df['Survived'] == 1)]
print(f"\n Kurtulan birinci sınıf sayısı: {len(kurtulan_birinci_sinif)}")


# isin() kullanarak liste bazlı filtreleme
# Biniş limanı C veya Q olan yolcuların Age ve Fare sütunlarını çeker
liman_cq = df.loc[df['Embarked'].isin(['C', 'Q']), ['Age', 'Fare']]
print("\n C ve Q limanından binen ilk 3 yolcunun yaş ve bilet ücreti: ")
display(liman_cq.head(3))
```

```
iloc ile ilk 5 satır 3 sütun:
```

PassengerId	Survived	Pclass	Name
1	0	3	Braund, Mr. Owen Harris
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...
3	1	3	Heikkinen, Miss. Laina
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)
5	0	3	Allen, Mr. William Henry

```
Kurtulan birinci sınıf sayısı: 136
 C ve Q limanından binen ilk 3 yolcunun yaş ve bilet ücreti:
```

PassengerId	Age	Fare
2	38.0	71.2833
6	NaN	8.4583
10	14.0	30.0708

iloc: İndeks sırasına göre ilk 5 satır ve ilk 3 sütun matris düzeyinde ayrıştırılmıştır.

loc & Boolean Indexing: 1. sınıfta seyahat edip hayatta kalan yolcular sorgulanmış ve 1. sınıftaki yolculardan tam 136 kişinin kurtulduğu doğrulanmıştır.

isin: Cherbourg (C) ve Queenstown (Q) limanlarından binen yolcular filtre edilerek yaş ve bilet ücretleri karşılaştırılmıştır.

VERİ TEMİZLEME VE DÖNÜŞTÜRME

```
# Orijinal veriyi bozmamak için kopya üzerinde çalışıyoruz
islenmis_df = df.copy()

# 1. Yaş sütunundaki eksik değerleri yaşın medyanı (28.0) ile dolduruyoruz
yas_medyani = islenmis_df['Age'].median()
islenmis_df['Age'] = islenmis_df['Age'].fillna(yas_medyani)

# 2. Cabin sütunundaki eksik değerlere Unknown atıyoruz
islenmis_df['Cabin'] = islenmis_df['Cabin'].fillna("Unknown")

# 3. Biniş limanı yani Embarked sütunundaki 2 eksik değeri en sık tekrar eden liman (Mod = 'S') ile dolduruyoruz
en_cok_binilen_liman = islenmis_df['Embarked'].mode()[0]
islenmis_df['Embarked'] = islenmis_df['Embarked'].fillna(en_cok_binilen_liman)

# 4. Kategorik değerleri Türkçeleştirme
islenmis_df['Sex'] = islenmis_df['Sex'].replace({'female': 'Kadin', 'male': 'Erkek'})

# 5. Sınıf sütununu sayısal (1, 2, 3) değil de bir kategori/etiket olarak algılaması için string yapıyoruz
islenmis_df['Pclass'] = islenmis_df['Pclass'].astype(str)

# Kontrol: Kalan eksik değer var mı? (Çıktının 0 vermesi beklenir)
print("İşlem Sonrası Toplam Eksik Değer Sayısı:", islenmis_df.isnull().sum().sum())
```

Windows

```
İşlem Sonrası Toplam Eksik Değer Sayısı: 0
```

Eksik Verilerin Giderilmesi: Age medyan ile doldurulmuş, Cabin için "Unknown", Embarked için 'S' atanarak toplam eksik veri sayısı 0'a indirilmiştir.

Vektörel Merkezileştirme: Bilet ücretlerinden genel ortalama çıkarılarak dağılım sıfır merkezine çekilmiştir.

Özellik Mühendisliği (Yolcu_Grubu): Yaş ve cinsiyet birleştirilerek modele anlamlı bir demografik etiket kazandırılmıştır: 519 Yetişkin Erkek, 259 Yetişkin Kadın ve 113 Çocuk grubu üretilmiştir.

HARİTALAMA VE MERKEZİLEŞTİRME

```
#1.Vektörel Merkezileştirme
# Bilet ücretlerinin (Fare) genel aritmetik ortalamasını hesaplıyoruz
ortalama_ucret = islenmis_df['Fare'].mean()

#bilet ücretinden ortalama ücreti çıkararak dağılımı 0 merkezine çekiyoruz
# ortalamadan pahalı biletler pozitif, ucuz biletler negatif değer alır
islenmis_df['Centered_Fare'] = islenmis_df['Fare'] - ortalama_ucret

#2.Yolcu Gruplama
#yaş ve cinsiyete göre yolcunun hangi grupta yer alacağını belirleyen fonksiyon
def yolcu_grubu(satir):
    if satir['Age'] < 18:
        return 'Cocuk'
    elif satir['Sex'] == 'Kadin':
        return 'Yetiskin_Kadin'
    else:
        return 'Yetiskin_Erkek'

# yolcu_grubu fonksiyonunu DataFrame'in tüm satırlarına uygular
# axis=columns ,satır satır çalışıp ilgili satırın sütun değerlerine erişim sağlar
islenmis_df['Yolcu_Grubu'] = islenmis_df.apply(yolcu_grubu, axis='columns')
```

```
# 3. Çıktıların İncelenmesi ve Kontrolü
print("Yeni Yolcu Grubu Dağılımı:")
print(islenmis_df['Yolcu_Grubu'].value_counts())

#merkezileştirmenin doğruluğunu kontrol etme
print(f"\nCentered_Fare Yeni Ortalaması: {islenmis_df['Centered_Fare'].mean():.4f}")

#oluşturulan yeni sütunların ilk 5 örneği
print("\nYeni Özelliklerin İlk 5 Gözlemi:")
display(islenmis_df[['Age', 'Sex', 'Fare', 'Centered_Fare', 'Yolcu_Grubu']].head())
```

```
Yeni Yolcu Grubu Dağılımı:
Yolcu_Grubu
Yetiskin_Erkek    519
Yetiskin_Kadin    259
Cocuk             113
Name: count, dtype: int64

Centered_Fare Yeni Ortalaması: -0.0000
```

Yeni Özelliklerin İlk 5 Gözlemi:

PassengerId	Age	Sex	Fare	Centered_Fare	Yolcu_Grubu
1	22.0	Erkek	7.2500	-24.954208	Yetiskin_Erkek
2	38.0	Kadin	71.2833	39.079092	Yetiskin_Kadin
3	26.0	Kadin	7.9250	-24.279208	Yetiskin_Kadin
4	35.0	Kadin	53.1000	20.895792	Yetiskin_Kadin
5	35.0	Erkek	8.0500	-24.154208	Yetiskin_Erkek

Feature / Target Ayrımı ve Model Öncesi Son Kontrol

07

```
#değişkenler belirleniyor
# 1. Modelin tahmin etmesini istediğimiz bağımlı değişkeni seçiyoruz
# 'Survived' sütunu yolcunun hayatta kalıp kalmadığını (0 öldü, 1 kurtuldu) temsil eder
y = islenmis_df['Survived']

# 2. Girdiler (Features) seçiliyor
# Modeli yolcunun durumunu tahmin etmek için kullanacağımız bağımsız değişkenleri seçiyoruz
# Yolcu ID, İsim ve Bilet numarası gibi doğrudan katkısı olmayan metin sütunlarını dahil etmiyoruz
X = islenmis_df[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']]

print("\nModel Öncesi Kontrol ")

# 1- Hedef değişkenin sınıf dağılımı
# normalize=True kullanarak hayatta kalanlar ile hayatını kaybedenlerin oranını yüzde olarak görüyoruz
# Amacı: Veri setinde aşırı sınıf dengesizliği olup olmadığını anlamak
# (Titanic için ~%61 Öldü [0], ~%39 Kurtuldu [1] dengeli ve öğrenmeye uygun bir dağılımdır)
print("1. y Dağılımı (Sınıf Dengesi Kontrolü):")
print(y.value_counts(normalize=True).round(4) * 100)

# 2- Girdi matrisinin (X) boyutu
# X.shape çıktısı (satır sayısı, sütun sayısı) formatında gelir
# Amacı seçilen özellik sayısını doğrulamak ve satır kaybı olup olmadığını kontrol etmektir
print(f"\n2. Girdi Matrisi X Boyutu: {X.shape}")

# 3- Eksik veri kontrolü
# X.isnull().sum() her sütundaki eksik değer sayısını verir, ikinci .sum() ise tüm tablodaki toplam eksik hücreyi sayar
# Amacı çoğu makine öğrenmesi algoritması NaN değerlerle çalışamaz. Çıktının 0 olması model öncesi verinin temiz olduğunu garantiler
print(f"\n3. X İçinde Kalan Eksik Veri Var Mı?: {X.isnull().sum().sum()}")
```

```
Model Öncesi Kontrol
1. y Dağılımı (Sınıf Dengesi Kontrolü):
Survived
0    61.62
1    38.38
Name: proportion, dtype: float64

2. Girdi Matrisi X Boyutu: (891, 7)

3. X İçinde Kalan Eksik Veri Var Mı?: 0
```

- Hedef Değişken (y): Modelin tahmin etmesi istenen bağımlı değişken orijinal Survived olarak belirlenmiştir (%61.62 Ölen [0], %38.38 Kurtulan [1]).
- Girdi Matrisi (X): Modele katkısı olmayan isim, bilet no ve ID elenmiş. 7 bağımsız öznitelikten oluşan (891, 7) boyutunda matris yapılandırılmıştır.
- Model Hazırlık Garantisi: Yapılan veri temizliği sayesinde X içindeki eksik hücre sayısı 0 olarak doğrulanmış ve algoritmaların hata almadan çalışması sağlanmıştır.

Bir Veri Setinde Model Kurmadan Önce Neleri Kontrol Etmeliyiz?

1- Boyut ve veri yapısı kontrolü.

2-Veri tipleri ve uygun kategori dönüşümleri.

3-Eksik veri ve temizleme stratejisi.

4-Özet istatistikler ve aykırı değer tespiti.

5-Hedef değişken sınıf dengesi (Target balance).



—04.11.2026—

TEŞEKKÜRLER

İlayda Bayrak

